

Algorytm generujący liczby Armstronga

Kacper Korzekwa, gr II

9 kwietnia 2026

1 Wprowadzenie

Liczba Armstronga to liczba naturalna, która jest sumą swoich cyfr podniesionych do potęgi ilości cyfr danej liczby. Problemem, nad którego rozwiązaniem się skupiłem, było zoptymalizowanie algorytmu tak, aby zajmował jak najmniej zasobów oraz generował jak najwięcej liczb w jak najkrótszym czasie.

2 Uzyskane algorytmy

2.1 Podejście 1: Metoda konwersji na ciąg znaków

W pierwszym podejściu skupiłem się na zamianie badanej liczby na ciąg znaków, dzięki czemu mogłem w prosty sposób wyodrębnić poszczególne cyfry oraz zliczyć ich ilość. Pozwoliło mi to na wygodne wykonanie wymaganych obliczeń dla każdej liczby po kolei w pętli. Niestety ciągłe zmiany typów dla każdej z liczb okazały się zbyt czasochłonne, dlatego po liczbie 24678051 algorytm mocno spowolnił. Skłoniło mnie to do zmiany podejścia w sposobie, w jaki rozkładałem każdą liczbę na pojedyncze cyfry w celu jej sprawdzenia.

2.1.1 Liczby wygenerowane w ciągu godziny:

Po ostatniej z podanych liczb w programie wystąpił błąd i przestał on poprawnie wskazywać liczby armstronga.

0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 153, 370, 371, 407, 1634, 8208, 9474, 54748, 92727, 93084, 548834, 1741725, 4210818, 9800817, 9926315, 24678050, 24678051, 88593477, 146511208, 472335975, 534494836, 912985153

2.1.2 Kod:

```
1 int check_if_armstrong(int test_number) {
2
3     if (test_number < 0) {
4         return 0;
5     }
6
7     std::string number_as_string = std::to_string(test_number);
8     int number_len{static_cast<int>(number_as_string.length())};
9     int res{0};
10
11     for (int i = 0; i < number_len; i++) {
12         int x = number_as_string.at(i) - '0';
13         res += std::pow(x, number_len);
14     }
15
16     return res == test_number;
17 }
```

2.2 Podejście 2: Wyciąganie poszczególnych cyfr za pomocą reszty z dzielenia

W drugim podejściu skupiłem się na dostaniu się do poszczególnych cyfr liczby wykorzystując resztę z dzielenia (modulo). Opierając się na czystej matematyce oraz tworząc kopie zadanej liczby w celu obliczenia jej długości, byłem w stanie zaoszczędzić dużo czasu, sprowadzając całość do

prosty działań arytmetycznych. Zmiany w wydajności są zauważalne natychmiastowo – w momentach, gdzie wcześniejszy algorytm widocznie zwalniał, nowa wersja cały czas utrzymywała wysokie tempo.

2.2.1 Liczby wygenerowane w ciągu godziny:

0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 153, 370, 371, 407, 1634, 8208, 9474, 54748, 92727, 93084, 548834, 1741725, 4210818, 9800817, 9926315, 24678050, 24678051, 88593477, 146511208, 472335975, 534494836, 912985153, 4679307774

2.2.2 Kod:

```
1  int64_t my_pow(int64_t num, int64_t size) {
2      int state{1};
3      int64_t res{num};
4
5      while (state < size)
6      {
7          res *= num;
8          state += 1;
9      }
10
11     return res;
12 }
13
14 int check_if_armstrong_v2(int64_t given_number) {
15     int64_t test_number = given_number;
16     if (test_number < 0)
17     {
18         return 0;
19     }
20
21     int size{0};
22     int64_t test_number_copy{test_number};
23
24     while (test_number_copy != 0)
25     {
26         test_number_copy = test_number_copy / 10;
27         size += 1;
28     }
29
30     int64_t res{0};
31     int64_t curr{};
32
33     while (test_number != 0)
34     {
35         curr = test_number % 10;
36         test_number = test_number / 10;
37         res += my_pow(curr, size);
38     }
39
40     return res == given_number;
41 }
```

3 Funkcja main sprawdzająca każdą liczbę:

```
1
2  int main() {
3
4      int check{};
5      bool working{true};
6      int64_t curr{0};
7
8      while (working == true)
9      {
10         check = check_if_armstrong_v2(curr);
11
12         if (check == 1) {
13             std::cout << curr << std::endl;
14         }
15     }
```

```
16         curr += 1;
17     }
18 }
19
```
